

LSM Genomic Parsing Failure and Recovery Postmortem

MCC HPC - genome_main recovery

Executive summary. The original genomic LSM job ran for roughly 48 hours and failed near 98% of column-group parsing. The immediate failure was an attempt to parse an empty JSON shard, 33.json. Investigation showed that the old shard-writing implementation repeatedly opened and rewrote the complete shard directly, creating a failure mode in which the existing file could be truncated before the replacement JSON was safely written. Recovery therefore reconstructs shard 33 from its beginning, starts parsing at column group 1650 rather than group 0, cleans stale header-index records, and uses a buffered/temporary-file shard writer.

1. Original failure

The original job reached approximately **1665 of 1699 column groups** and then terminated with a nlohmann JSON parse error: attempting to parse an empty input. Slurm showed the job as FAILED, but the evidence did not indicate a normal time-limit or memory exhaustion: the requested limit was 80 hours and observed memory was below the 50 GB request.

```
Fatal: [json.exception.parse_error.101]
parse error at line 1, column 1:
attempting to parse an empty input
```

2. Direct cause: an empty shard file

Inspection of the generated source maps found a zero-byte file at results/genome_main/source_maps/json_shards/33.json. The source-map reader obtains the file size, creates a string of that size, reads the bytes, and passes the string to json::parse. For a zero-byte file the string is empty, which directly produces the observed parse error.

```
33.json -> 0 bytes
empty file -> empty buffer -> json::parse(empty buffer) -> parse_error.101
```

3. Why shard 33 was involved

The project uses COLUMNS_PER_SHARD = 50000. Therefore shard 32 contains columns 1,600,000-1,649,999 and shard 33 begins at column 1,650,000. Each parser column group contains 1000 columns. The failed job had reached the group-1665 region, and the header index later showed records through column 1,665,511. Those facts align the failure with shard 33.

4. Problem in the previous ShardMap code

The older implementation loaded the existing shard, added one column, then directly opened the final shard path and rewrote the complete JSON. Opening the destination this way truncates the existing file before the new contents are fully written. If the process fails after truncation but before a complete write, a previously valid shard can be left empty or incomplete. It also means an increasingly large JSON document is read and rewritten for every individual column.

```
nlohmann::json shardJson;
if (std::filesystem::exists(shardFile)) {
    std::ifstream in(shardFile);
    if (in) in >> shardJson;
}
...
shardJson[std::to_string(column_id)] = std::move(colObj);

std::ofstream out(shardFile);
out << shardJson.dump();
```

5. Backups and preservation

Before modifying the generated data, the complete `source_maps` directory was copied to `source_maps_BACKUP`. The original executable was also preserved as `bin/LSM.pre_recovery`. This kept the failed state and the original September 20 executable available for comparison or rollback.

6. Recovery boundary: start at group 1650

The corrupt shard could not simply be resumed at group 1665 because shard 33 itself begins at column 1,650,000, corresponding to group 1650. Since the entire shard file was lost, recovery must rebuild shard 33 from its beginning. The parser was therefore changed from group index 0 and starting column 0 to group index 1650 and starting column $1650 * \text{COLUMN_GROUP_SIZE}$. This regenerates groups 1650-1698 - only 49 groups - instead of repeating all 1699 groups.

```
// Before
int columnGroupIndex = 0;
for (int startingColumn = 0; ...)

// Recovery
int columnGroupIndex = 1650;
for (int startingColumn = 1650 * COLUMN_GROUP_SIZE;
startingColumn < CurrentFileContext.numColumns;
startingColumn += COLUMN_GROUP_SIZE) {
```

7. Separate source-code mismatch discovered during recovery

The project copy of `src/ShardMap/ShardMap.cpp` was found to be zero bytes. An older 110-line copy was recovered from a nearby directory, but it did not match the current header. The header declares a destructor, `flush()`, `switchShard()`, `flushCurrentShard()`, and buffered shard state. The older source did not implement these members, producing the linker error `undefined reference to shardio::ShardMap::~ShardMap()`.

8. ShardMap implementation reconstructed

The implementation was reconstructed to match the current header. It now keeps one shard in memory using `currentShardJson_`, tracks the current shard ID and dirty state, and flushes when switching shards or when the object is finalized. This avoids reading and rewriting the same shard once for every column and matches the design already represented in the header.

9. Safer shard writing

The new flush path writes the complete JSON to a temporary file first, flushes/closes it, and then renames it to the final shard path. Thus the final shard is not intentionally truncated before a complete replacement exists. For the recovery run, shard 33 and any stale `33.json.tmp` were confirmed absent before submission.

```
write complete JSON -> 33.json.tmp
flush/close temporary file
rename 33.json.tmp -> 33.json
```

10. Header-index cleanup

The header index is append-only. The failed run had already appended entries for part of shard 33, so rerunning from group 1650 without cleanup would create duplicates. Measurement showed 1,665,512 total records, with 15,512 records at column IDs $\geq 1,650,000$ and a maximum ID of 1,665,511. Those 15,512 records were removed. Verification then showed zero records $\geq 1,650,000$ and a maximum remaining column of 1,649,999.

11. Build result

After reconstructing `ShardMap.cpp`, the Release target linked successfully. The new executable is approximately 594 KB with the September 29 timestamp, while the preserved original is approximately 584 KB with the September 20 timestamp.

```
cmake --build build-bindings --target LSM -j 4
[2/2] Linking CXX executable ../bin/LSM
```

12. What the recovery job does now

The recovery job still performs the parser's initial row-processing/setup phase because only the later column-group loop was changed. This is expected. When it reaches column-group parsing, it should begin at group 1650 and regenerate groups 1650-1698. Because the new ShardMap buffers the shard, 33 . json may not appear while individual groups are being processed; it is written when the shard is flushed.

Summary of direct causes and fixes

Problem	Direct cause	Change made
Original runtime crash	Shard 33 was zero bytes, so JSON parsing received empty input.	Remove and reconstruct shard 33 from its beginning.
Shard corruption risk	Old code directly truncated and rewrote the final shard for every column.	Buffer a shard and write a complete temporary file before rename.
Recovery build linker failure	Recovered 110-line ShardMap.cpp did not match the current ShardMap.hpp interface.	Reconstruct destructor, flush/switch logic, buffered state, and append functions.
Potential duplicate header records	Failed run had already appended 15,512 shard-33-range header-index records.	Remove all header-index entries with col >= 1,650,000 before rerun.
Avoid repeating 48-hour parse	Original loop always started at group 0.	Start recovery at group 1650 and rebuild only groups 1650-1698.

Current status: the repaired binary builds successfully, shard 33 is being regenerated from group 1650 onward, and the initial row-processing phase seen at job startup is expected.